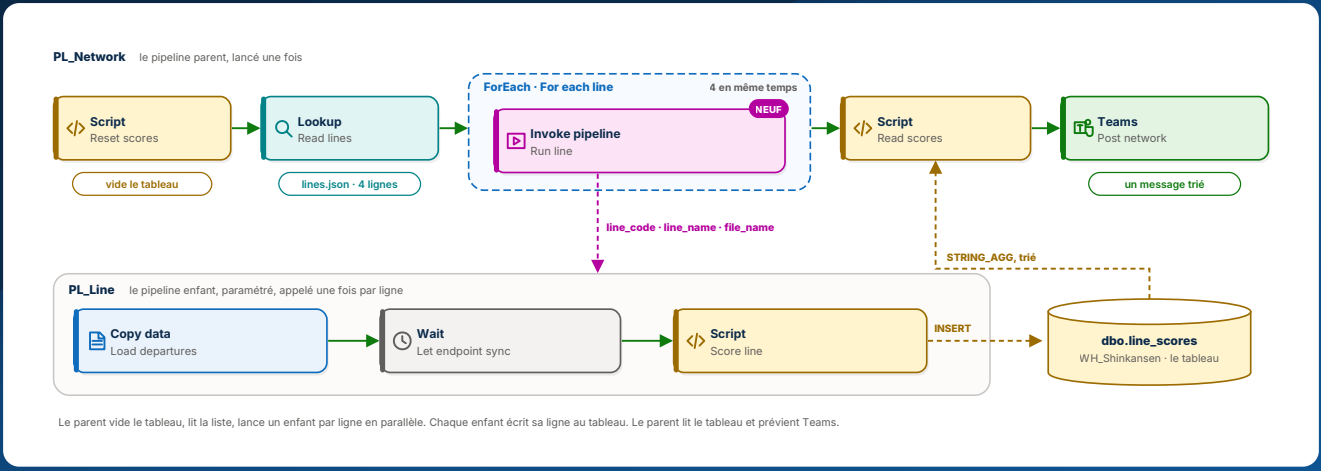
MICROSOFT FABRIC · DATA FACTORY · WAREHOUSE · GUIDE DE RÉFÉRENCE

Pipeline Microsoft Fabric : un parent, un enfant, quatre lignes

PRÉPARER LIRE DÉLÉGUER CHARGER MESURER PRÉVENIR

Japon, mardi 20 octobre 2026. Quatre lignes de Shinkansen déposent leurs départs du jour. Un pipeline parent lit la liste des lignes dans un fichier JSON et lance, pour chacune et en même temps, un pipeline enfant paramétré. Chaque enfant charge sa ligne, mesure sa ponctualité et inscrit son verdict au tableau. Le parent lit le tableau et annonce le classement dans Teams.



9	2	4	358	0
activités	pipelines	enfants en parallèle	départs	ligne écrite en dur

AUTEUR
Abdoul Hamid Diallo, Microsoft Data & AI Engineer

PRÉREQUIS
Un espace de travail Fabric sur une capacité, Microsoft Teams avec une équipe

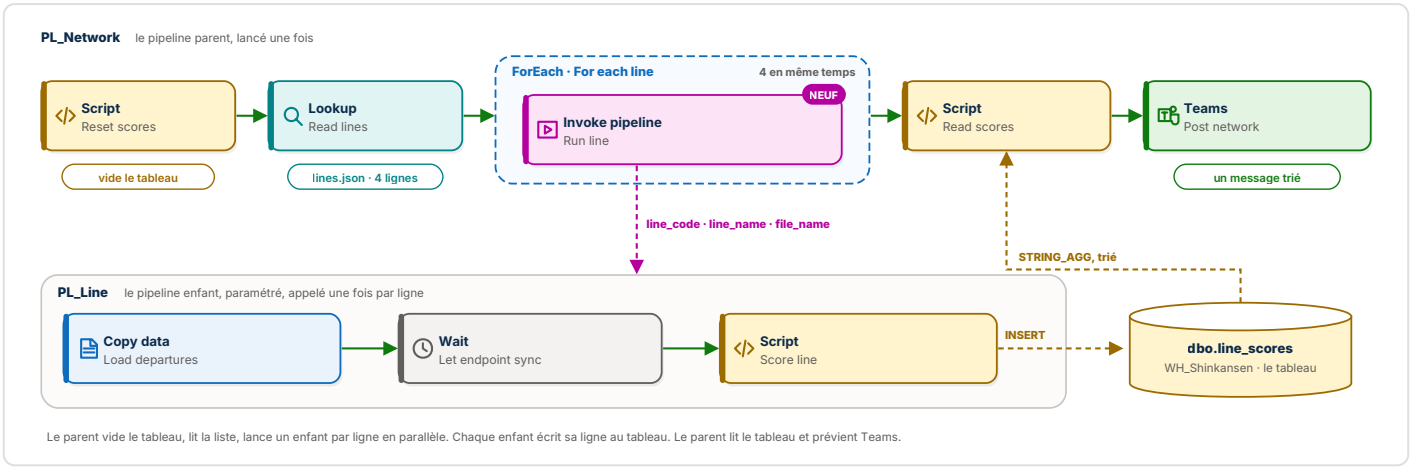
FICHIERS DU PROJET
Quatre CSV de départs, lines.json, expressions.txt

DURÉE
Environ 30 minutes, en quatre étapes vérifiées une à une

VUE D'ENSEMBLE

Ce que vous allez construire

Tōkaidō, Sanyō, Tōhoku, Hokkaidō : chaque ligne envoie un CSV de ses départs, avec le retard de chaque train en secondes. Je pourrais écrire un seul gros pipeline qui connaît les quatre lignes par cœur. Je vous propose mieux : **deux petits pipelines**. L'enfant sait traiter **une** ligne, n'importe laquelle. Le parent sait **combien** il y en a et lesquelles, parce qu'il le lit dans un fichier. C'est le modèle qu'on retrouve dans les plateformes de données en production.



Le pipeline en une image : une gare

La feuille de service
lines.json dit quelles lignes contrôler ce jour-là.

Le chef de gare
PL_Network lit la feuille et envoie un contrôleur par ligne.

Les contrôleurs
PL_Line, lancé quatre fois en même temps, chacun sur sa ligne.

Le tableau
line_scores : chaque contrôleur y écrit sa ligne.

L'annonce
Teams lit le tableau, du plus ponctuel au moins ponctuel.

Les activités

ACTIVITÉ	TYPE	RÔLE
PL_Line, l'enfant : trois paramètres, trois activités		
`Load departures`	Copy data	Charge le CSV reçu en paramètre dans la table `dep_<ligne>`
`Let endpoint sync`	Wait	Laisse au point de terminaison SQL le temps de voir la nouvelle table
`Score line`	Script	Calcule la ponctualité et inscrit une phrase verdict dans `line_scores`
PL_Network, le parent : six activités		
`Reset scores`	Script	Efface le tableau de la veille
`Read lines`	Lookup	Lit `lines.json` : la liste des lignes à traiter
`For each line`	ForEach	Une itération par ligne, quatre en même temps
`Run line`	Invoke pipeline NEUF	Lance `PL_Line` avec le code, le nom et le fichier de la ligne
`Read scores`	Script	Assemble le tableau en une phrase, lignes triées
`Post network`	Microsoft Teams	Publie le classement du réseau

CE QUE VOUS APPRENEZ Les **paramètres de pipeline**, l'**orchestration parent et enfant** avec **Invoquer le pipeline**, le **ForEach en parallèle**, une liste **pilotée par un fichier JSON** et une **requête interbases** du Warehouse vers le Lakehouse.

LE BÉNÉFICE Une cinquième ligne demain ? Une entrée de plus dans `lines.json` et un CSV de plus. Aucun pipeline n'est ouvert, aucune activité n'est copiée.

ÉTAPE 1

Le lakehouse, le warehouse, le tableau

1 · OBJETS | 2 · L'ENFANT CHARGE | 3 · L'ENFANT MESURE | 4 · LE PARENT ORCHESTRE

1 LH_Shinkansen — LAKEHOUSE

1. Dans votre espace de travail, **Nouvel élément**, **Lakehouse**, nom `LH_Shinkansen`.
2. Dans **Fichiers**, **Nouveau sous-dossier** : `inbox`, puis un second : `config`.
3. Dans `inbox`, **Charger**, **Charger des fichiers** : les **quatre** CSV, en une fois. Dans `config` : `lines.json`.

2 WH_Shinkansen et son tableau — WAREHOUSE

1. **Nouvel élément**, **Entrepôt** (Warehouse), nom `WH_Shinkansen`.
2. Dans l'entrepôt, **Nouvelle requête SQL**, puis **Exécuter** :

```
CREATE TABLE dbo.line_scores (line_code
varchar(20), on_time_pct decimal(5,1), summary
varchar(200));
```

LE BON ENDROIT Vérifiez que l'explorateur affiche `WH_Shinkansen` en haut à gauche. Le point de terminaison SQL du lakehouse est en lecture seule : il refuse tout `CREATE TABLE`.

LIGNE	TRAJET	DÉPARTS
Tōkaidō	Tokyo, Shin-Osaka	132
Sanyō	Shin-Osaka, Hakata	96
Tōhoku	Tokyo, Shin-Aomori	104
Hokkaidō	Shin-Aomori, Shin-Hakodate-Hokuto	26

Colonnes de chaque CSV : `train_id`, `train_type`, `origin`, `destination`, `sched_dep`, `delay_s`. Données fictives et réalistes.

`lines.json`, le fichier qui pilote tout :

```
[
  {"code": "tokaido", "name": "Tōkaidō",
   "file": "departures_tokaido_2026-10-20.csv"},
  {"code": "sanyo", "name": "Sanyō",
   "file": "departures_sanyo_2026-10-20.csv"},
  {"code": "tohoku", "name": "Tōhoku",
   "file": "departures_tohoku_2026-10-20.csv"},
  {"code": "hokkaido", "name": "Hokkaidō",
   "file": "departures_hokkaido_2026-10-20.csv"}
]
```

Le lakehouse stocke

Les fichiers bruts et les tables Delta. Son point de terminaison SQL se lit, il ne s'écrit pas.

Le warehouse calcule et écrit

Il lit `LH_Shinkansen.dbo.dep_tokaido` par son nom à trois parties, sans rien copier, et écrit dans `line_scores`.

Le JSON décide

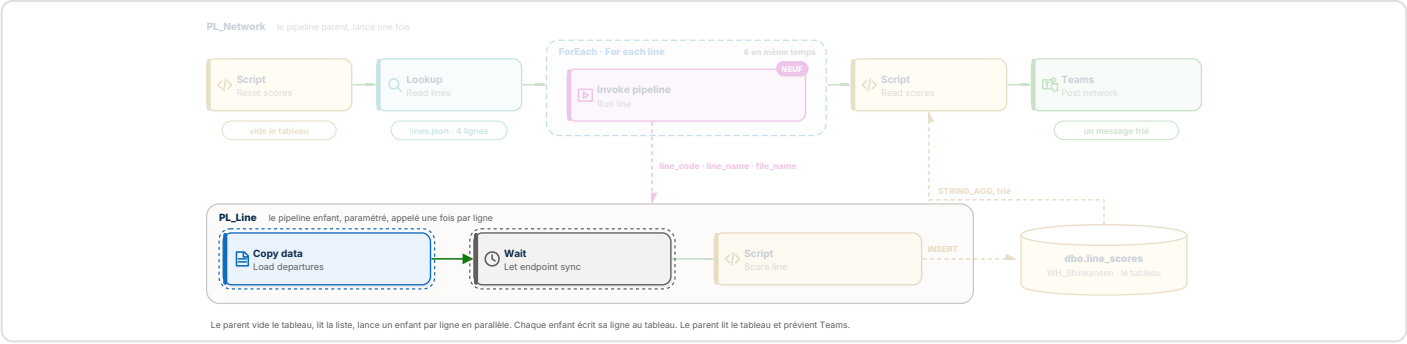
Aucune ligne n'est écrite dans les pipelines. Toute la liste vit dans un fichier.

AVANT LES PIPELINES ☐ 4 CSV dans `inbox` ☐ `lines.json` dans `config` ☐ `line_scores` créée ☐ tout dans le même espace

ÉTAPE 2

L'enfant : trois paramètres, un chargement

1 · OBJETS | 2 · L'ENFANT CHARGE | 3 · L'ENFANT MESURE | 4 · LE PARENT ORCHESTRE



3 PL_Line et ses paramètres

PIPELINE ENFANT

1. **Nouvel élément**, **Pipeline**, nom `PL_Line`.
2. Cliquez dans le vide du canevas, onglet **Paramètres**, **+ Nouveau**, trois fois. Les valeurs par défaut servent au test de l'étape 3.

NOM	TYPE	VALEUR PAR DÉFAUT	RÔLE
`line_code`	String	`tokaido`	Nom de la table, sans accent
`line_name`	String	`Tōkaidō`	Nom affiché dans le verdict
`file_name`	String	`departures_tokaido_2026-10-20.csv`	Fichier à charger

4 Load departures, source

COPY DATA

Connexion	`LH_Shinkansen`
Racine	`Fichiers`
Type de chemin	Chemin d'accès au fichier
Dossier	`inbox`
Nom de fichier	Ajouter du contenu dynamique :

```
@pipeline().parameters.file_name
```

Format	DelimitedText, Première ligne comme en-tête coché

Load departures, destination

Connexion	`LH_Shinkansen`
Racine	`Tables`
Table	Ajouter du contenu dynamique :

```
dep_@{pipeline().parameters.line_code}
```

Action de table	`Remplacer` (Overwrite)

5 Let endpoint sync

WAIT

Attendre, nom `Let endpoint sync`, relié à la sortie verte de `Load departures`. **Temps d'attente en secondes** : `90`.

UN PARAMÈTRE, C'EST LA PORTE D'ENTRÉE Un paramètre est fixé **au lancement** et ne change plus pendant l'exécution. Le parent y glissera le code, le nom et le fichier de chaque ligne : le même enfant sert pour les quatre.

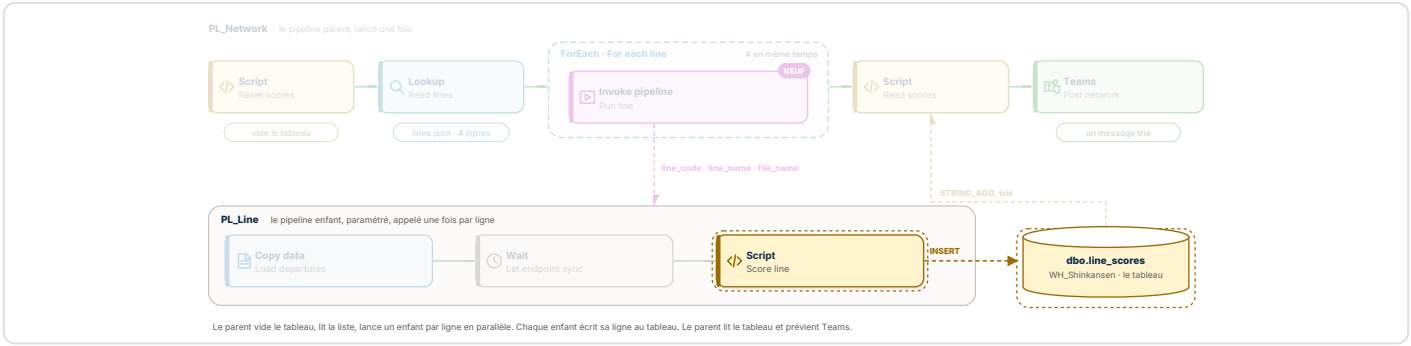
POURQUOI ATTENDRE 90 S Une table Delta toute neuve met un moment à apparaître dans le point de terminaison SQL. Le Script de l'étape 3 la lit par ce point de terminaison : la pause lui garantit de la trouver.

LE TEXTE ET LE DYNAMIQUE Tapé tel quel, `dep_@{…}` deviendrait le nom littéral de la table. C'est le bouton **Ajouter du contenu dynamique** qui le transforme en `dep_tokaido`.

ÉTAPE 3

L'enfant mesure et écrit au tableau

1 · OBJETS | 2 · L'ENFANT CHARGE | 3 · L'ENFANT MESURE | 4 · LE PARENT ORCHESTRE



6 Score line

SCRIPT SUR LE WAREHOUSE

1. **Script**, nom `Score line`, relié à la sortie verte de `Let endpoint sync`.
2. **Paramètres** : connexion `WH_Shinkansen`, type de script **Requête**. Dans le champ du script, **Ajouter du contenu dynamique**, puis collez :

```
INSERT INTO dbo.line_scores SELECT '@{pipeline().parameters.line_code}', CAST(100.0 * SUM(CASE WHEN CAST(delay_s AS int) < 60 THEN 1 ELSE 0 END) / COUNT(*) AS decimal(5,1)), CONCAT('@{pipeline().parameters.line_name} ', CAST(100.0 * SUM(CASE WHEN CAST(delay_s AS int) < 60 THEN 1 ELSE 0 END) / COUNT(*) AS decimal(5,1)), ' % à l''heure (', COUNT(*), ' départs, max ', MAX(CAST(delay_s AS int)), ' s)') FROM LH_Shinkansen.dbo.dep_@{pipeline().parameters.line_code}; SELECT 1 AS ok;
```

Lire la requête en quatre morceaux

1. `FROM LH_Shinkansen.dbo.dep_@{…}` : le warehouse lit la table de la ligne dans le lakehouse, par son nom à trois parties.
2. `CASE WHEN delay_s < 60` : un train est à l'heure s'il part avec moins d'une minute de retard. La somme divisée par `COUNT(*)` donne le pourcentage.
3. `CONCAT(…)` : assemble la phrase verdict. `l''heure` : deux apostrophes pour en écrire une en SQL.
4. `SELECT 1 AS ok` : une petite réponse, parce que le type **Requête** attend un résultat.

UN INSERT, JAMAIS UN UPDATE Les quatre enfants écriront en même temps dans `line_scores`. Des insertions simultanées dans une même table du warehouse cohabitent sans conflit.

Exécution 1 — l'enfant seul, sur Tōkaidō

Enregistrer, **Exécuter**. La fenêtre propose les trois paramètres déjà remplis : **OK**. Trois activités vertes. Puis, dans `WH_Shinkansen` :

```
SELECT * FROM dbo.line_scores;
```

```
tokaido · 99.2 · Tōkaidō 99.2 % à l'heure (132 départs, max 131 s)
```

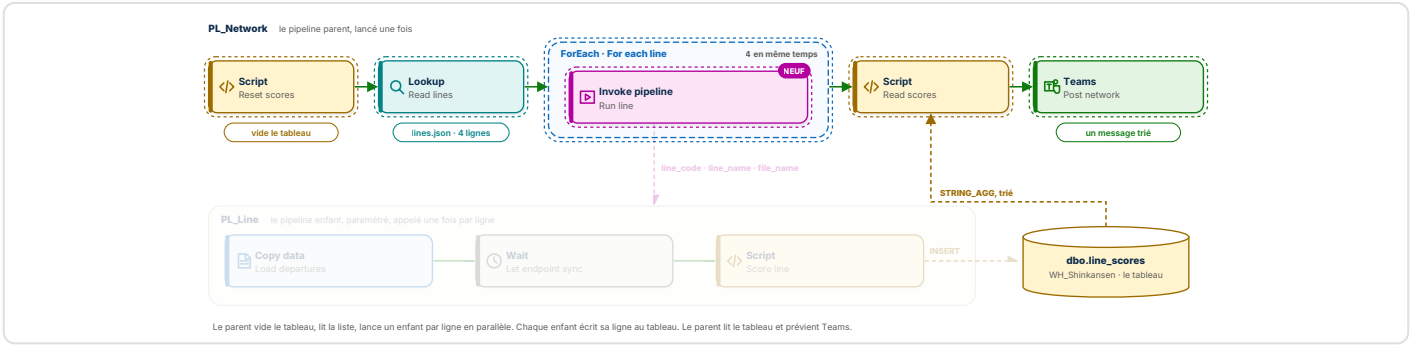
MÊME PIPELINE, AUTRE LIGNE Relancez avec `tohoku`, `Tōhoku`, `departures_tohoku_2026-10-20.csv`. Une deuxième ligne apparaît : 91.3 %, max 1100 s. Rien n'a été modifié dans le pipeline.

NE NETTOYEZ PAS Les lignes de test peuvent rester : le parent commence toujours par vider le tableau.

ÉTAPE 4

Le parent vide, lit, délègue, annonce

1 · OBJETS | 2 · L'ENFANT CHARGE | 3 · L'ENFANT MESURE | 4 · LE PARENT ORCHESTRE



7 Reset scores, Read lines

SCRIPT · LOOKUP

1. **Nouvel élément**, **Pipeline**, nom `PL_Network`.
2. **Script**, nom `Reset scores`. Connexion `WH_Shinkansen`, type **Requête** :

```
DELETE FROM dbo.line_scores; SELECT 1 AS ok;
```

3. **Rechercher**, nom `Read lines`, relié à `Reset scores`. Connexion `LH_Shinkansen`, **Fichiers**, dossier `config`, fichier `lines.json`, format **JSON**. **Première ligne uniquement** **décoché**.

8 For each line

FOREACH

ForEach, relié à `Read lines`. **Séquentiel** décoché, **Nombre de lots** : `4`. **Éléments** :

```
@activity('Read lines').output.value
```

9 Run line

INVOKE PIPELINE

1. Dans le ForEach (crayon), **Invoquer le pipeline**, nom `Run line`. Connexion : **Nouvelle**, **Compte d'organisation**. Votre espace, pipeline `PL_Line`, **Attendre la fin** coché.
2. Les trois paramètres de l'enfant apparaissent. Valeurs, dans l'ordre :

```
@item().code
```

```
@item().name
```

```
@item().file
```

10 Read scores

SCRIPT

Retour au canevas principal. **Script**, nom `Read scores`, relié à la sortie verte du ForEach. Connexion `WH_Shinkansen`, type **Requête** :

```
SELECT STRING_AGG(summary, ' | ') WITHIN GROUP (ORDER
BY on_time_pct DESC) AS report FROM dbo.line_scores;
```

11 Post network

MICROSOFT TEAMS

1. **Microsoft Teams**, relié à `Read scores`. Votre équipe, canal **Général**, **Titre** : `Shinkansen`.
2. **Message** :

```
Shinkansen, départs du 20 octobre 2026 :
@{activity('Read
scores').output.resultSets[0].rows[0].report}
```

Exécution 2 le réseau entier

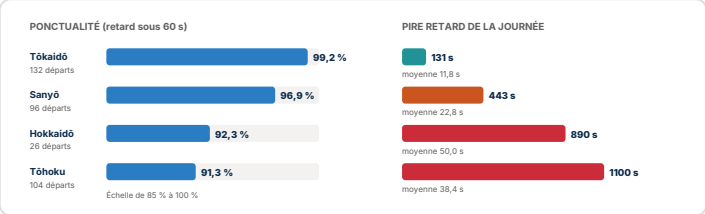
Enregistrer, **Exécuter**. Le Moniteur affiche **cinq** exécutions : `PL_Network` et quatre `PL_Line` lancées à la même seconde. Dans Teams, le classement arrive trié :



STRING_AGG Les enfants finissent dans n'importe quel ordre. `WITHIN GROUP (ORDER BY …)` remet le classement d'aplomb : du plus ponctuel au moins ponctuel.

RÉFÉRENCE

Lire le résultat, retenir le modèle, débloquer



358
départs sur quatre lignes

95,8 %
à l'heure sur le réseau (343 sur 358)

1100 s
pire retard, sur Tōhoku : 18 min 20 s

1 sur 132
seul départ en retard sur Tōkaidō

Ce qu'il faut retenir

L'enfant ne sait rien d'avance
Trois paramètres suffisent à le rendre universel. On le teste seul, ligne par ligne, avant de le confier au parent.

Le tableau relie les deux
Invoquer le pipeline rend au parent le statut de l'enfant : réussi ou échoué. Les données, elles, passent par une table partagée. C'est simple, lisible et vérifiable en SQL.

Vider, remplir, lire
`Reset scores` au début, `INSERT` dans chaque enfant, `STRING_AGG` à la fin. Le tableau reflète toujours la dernière exécution, et seulement elle.

Le fichier pilote
Ajouter, retirer ou renommer une ligne se fait dans `lines.json`, jamais dans le pipeline.

Si quelque chose bloque

SYMPTÔME	CAUSE ET CORRECTION
`Score line` : « Invalid object name … dep_tokaido »	Le point de terminaison SQL ne voit pas encore la table. Dans le lakehouse, **Point de terminaison SQL analytique**, **Actualiser**, puis relancez. L'attente de 90 s l'évite ensuite.
Le lakehouse montre une table `dep_@{pipeline()…}`	Le nom de table a été tapé comme du texte. Effacez-le, **Ajouter du contenu dynamique**, recollez. Supprimez la table fautive.
`CREATE TABLE` refusé	La requête est lancée sur le lakehouse. Ouvrez `WH_Shinkansen` et relancez-la là.
Un chiffre rouge sur l'onglet **Paramètres**	Un champ obligatoire est vide, souvent la connexion. **Accueil**, **Valider** nomme le champ exact.
`Read lines` ne renvoie qu'une ligne	**Première ligne uniquement** est resté coché. Décochez.
Teams montre plus de quatre lignes	`Reset scores` n'est pas relié à `Read lines`. Reliez sa sortie verte.
Échec « AADSTS700082 »	Connexion expirée. **Gérer les connexions** ou nouvelle connexion, **Compte d'organisation**.